# Radeon AI Creative Studio

Project Profile Document

### Track 1: Development of Multimodal Content Creation Tools

Local visual ideation powered by AMD Radeon GPU and ROCm

Prepared for the 2026 AMD AI DevMaster Radeon GPU Hackathon

# 1. Project background

Creative teams often need many visual directions before choosing one final design. Cloud-only image tools can add latency, usage cost, privacy concerns, and limited control over the execution environment. Radeon AI Creative Studio is a lightweight local visual ideation workstation that turns text ideas and reference images into a small set of editable creative candidates.

The project targets Track 1 and demonstrates text-to-image generation, reference-image restyling, batch ideation, and image enhancement in a single web interface.

# 2. Target users and application scenarios

• Social media creators preparing campaign concepts and post artwork.

• Small businesses exploring product, poster, and promotional visual directions.

• Visual designers generating reference variations before manual refinement.

• Content teams producing several seeded candidates from one idea.

The practical value is fast local exploration: a user can compare seeded alternatives, refine a reference image, and prepare a higher-resolution output without sending the core prompt and image data to a closed image-generation API.

# 3. Complete creation workflow

The application provides a complete input-processing-output loop through a Gradio web interface.

| Input | Processing | Output |
|---|---|---|
| Text idea or reference image | Prompt templates, Diffusers, PyTorch, ROCm | PNG candidates, history, download |
| Optional style and quality controls | Stable Diffusion text-to-image or image-to-image | Timing, seed, GPU and VRAM metadata |

# 4. Model and algorithm

The core model is stable-diffusion-v1-5/stable-diffusion-v1-5, loaded through Hugging Face Diffusers. StableDiffusionPipeline handles text-to-image generation and StableDiffusionImg2ImgPipeline handles reference-image conditioning.

• FP16 weights reduce memory pressure.

• torch.inference_mode() avoids unnecessary autograd work.

• Explicit seeds make output comparisons reproducible.

• Batch generation uses adjacent deterministic seeds for quick concept selection.

The enhancement module uses deterministic Pillow operations for auto-contrast, brightness, contrast, color, sharpness, Unsharp Mask detail enhancement, and Lanczos enlargement. This is CPU post-processing and is not presented as neural GPU inference.

# 5. AMD Radeon GPU and ROCm adaptation

The application is designed for the designated AMD Radeon Cloud environment. The ROCm-enabled PyTorch build exposes the available Radeon device through the CUDA-compatible PyTorch API used by Diffusers. At startup, the application checks torch.cuda.is_available(), detects the GPU name, reads total VRAM, and moves both diffusion pipelines to cuda.

### Key inference stages executed locally on the AMD Radeon GPU:

• Text encoding and diffusion denoising for text-to-image.

• Text encoding, denoising, and reference-image conditioning for image-to-image.

• Repeated or batched diffusion requests for batch creation.

The UI reports GPU name, VRAM, seed, image size, steps, and elapsed time after each core generation request. The final submission includes the exact Radeon Cloud container tag and measured timings captured from the verified run.

# 6. Deployment and reproducibility

The repository includes the Gradio application, Python modules, dependency list, startup command, model identifier, and reproducibility instructions. The Radeon Cloud container should provide ROCm-compatible PyTorch and torchvision wheels; the application requirements intentionally do not overwrite those container-specific wheels with generic PyPI packages.

The model is not bundled in the repository. It is loaded from the Hugging Face cache or downloaded on first use. For an offline evaluator run, the model should be pre-cached in the container before launching the application.

# 7. Limitations and future work

• The core pipeline requires a ROCm-compatible PyTorch build and an AMD Radeon GPU.

• The first model load may take several minutes.

• Large resolutions and large batches may exceed available VRAM.

• Future work includes a GPU-native super-resolution model, LoRA style packs, ControlNet guidance, and video clip stylization.

# 8. Demonstrated outputs

The repository contains curated examples from the verified development session. They represent the four user-facing creation modes.

### Text-to-image



### Image-to-image



### Batch candidate



### Enhanced output

